

Export_grade-Cryptohack

cryptography

Analisis

deskripsi chall=

Alice and Bob are using legacy codebases and need to negotiate parameters they both support. You've man-in-the-middle this negotiation step, and can passively observe thereafter. How are you going to ruin their day this time?

Connect at socket.cryptohack.org 13379

Enter flag here: crypto{FLAG}

ketika kita nc socket.cryptohack.org 13379 maka kita jadi orang yang ada di tengah tengah (man in the middle) di percakapan antara Alice dan Bob.

```
(siber@ITNSA)-[/mnt/d/py_repin]
$ nc socket.cryptohack.org 13379
Intercepted from Alice: {"supported": ["DH1536", "DH1024", "DH512", "DH256", "DH128", "DH64"]}
Send to Bob: |
```

di sana kita bisa mengirimkan opsi ke Bob untuk menentukan panjang dari nilai A dan B yang akan di sepakati Alice dan Bob. lalu Bob akan memilih pilihan nya, lalu kita kirim pilihan dari Bob ke Alice dan Alice akan memberikan A, p, g dimana p dan A akan menyesuaikan panjang dari pilihan Bob tadi (jika pilih DH1024 maka ukuran akan jadi 1024) lalu Bob memberikan B (dengan panjang) setelah itu Alice merespon lagi dengan memberi iv dan ct_flag.

```
(siber@ITNSA)-[/mnt/d/py_repin]
$ nc socket.cryptohack.org 13379
Intercepted from Alice: {"supported": ["DH1536", "DH1024", "DH512", "DH256", "DH128", "DH64"]}
Send to Bob: {"supported": ["DH1536", "DH1024", "DH512", "DH256", "DH128", "DH64"]}
Intercepted from Bob: {"chosen": "DH1024"}
Send to Alice: {"chosen": "DH1024"}
Intercepted from Alice: {"p": "0xf2639ce2b2b2e67154813bcbda8e5a09ddaa1235c5e76300602e29ada9dd6dfdf36b3c6a676891ddb1462de67cc27a45f84d8720b8bfcdb653c82814397998e84aafca63a8b4ae05d3193e7566173441d505dc3caea006f938d421de7e80748297496436e559fe9c443201de066cd7570a8a40c80a306309dfb4da48277858b", "g": "0x2", "A": "0xabbd8c8297896ef4ed476c03bac298556eda599bd8692a3e3eb5301dd1cad3bd2763920ab128542b93e2db324d870b063b450c8447b993681a908986022c6e00cab91c08d9181be17c5545d09288bafae6d84ace76b5f77d8df6937d4c40f216ddc8fb5986d18921073e5aca5a65080451fc87c4fd84c87303bd7f1de20739b8"}
Intercepted from Bob: {"B": "0x370719f48213736983709e554654c797ccf39bee19831f4e36902ef9842dd592ebfd810f09898efe7a1650a981d7ee398956fef32c4dfb0d79bd7aladc01b54676c2568fc66f4eed7641d2e5110e7ae4283dc338fb6f20eb75d2c8914d88a1dab9e41253ca0bc6681cdb5ec24e68329e3444475ce67acce05ccf5cb9a4cf5ac"}
Intercepted from Alice: {"iv": "69e88b4ed4aa6f6794f7078b2041bce1", "encrypted_flag": "e8362dc3987b26e2d0ac8bd5f10d5a33736d0385060b3bbbedff8f77f1453c6f6"}

```

Solusi

jika Bob memberikan respon misal DH1024 tapi kita malah mengirim DH64 ke Alice maka akan eror.

```
(siber@ITNSA)-[/mnt/d/py_repin]
$ nc socket.cryptohack.org 13379
Intercepted from Alice: {"supported": ["DH1536", "DH1024", "DH512", "DH256", "DH128", "DH64"]}
Send to Bob: {"supported": ["DH1536", "DH1024", "DH512", "DH256", "DH128", "DH64"]}
Intercepted from Bob: {"chosen": "DH1024"}
Send to Alice: {"chosen": "DH64"}
Intercepted from Alice: {"p": "0xde26ab651b92a129", "g": "0x2", "A": "0x6d1b1a0757937f82"}
Intercepted from Bob: {"error": "Prime unsupported"}

```

jadi kita harus membuat pilihan dari Bob cuma ada DH64. karena kita diperbolehkan server merubah rubah pesan sebelum di kirim ke Alice maka kita tinggal memanipulasi pilihan

untuk di kirim ke Bob jadi {"supported": "DH64"}.kenapa kita harus mengirim pilihan DH64 dan membuat p jadi sekecil mungkin ukurannya?supaya kita bisa mengcaculatkan A,g,p di **Discrete logarithm calculator** supaya menghasilkan a,dan dari a yang kita dapat dari **Discrete logarithm calculator** kita bisa dapat S supaya bisa mendekripsi kan flag.

solver =

```
print(len('6c4cdfdc516766c869c393e8f0f7c43d1f3ea16304573591c1a577351458330f'))

p=int("0xde26ab651b92a129",16)
B = int("0xc4277525b7d8ce32",16)
g = int("0x2",16)
A = int("0x3b0db8ebcfe70de8",16)
a = 5077382450032242995 #didapat dari disceret lagarithm calculator

print(f"p = {p}")
print(f"A = {A}")
print(f"g = {g}")

shared = pow(B,a,p)
print(shared)
```

solver untuk medekripsi kan flag =

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
import hashlib

def is_pkcs7_padded(message):
    padding = message[-message[-1]:]
    return all(padding[i] == len(padding) for i in range(0,
len(padding)))

def decrypt_flag(shared_secret: int, iv: str, ciphertext: str):
    # Derive AES key from shared secret
    sha1 = hashlib.sha1()
    sha1.update(str(shared_secret).encode('ascii'))
    key = sha1.digest()[:16]
```

```
# Decrypt flag
ciphertext = bytes.fromhex(ciphertext)
iv = bytes.fromhex(iv)
cipher = AES.new(key, AES.MODE_CBC, iv)
plaintext = cipher.decrypt(ciphertext)

if is_pkcs7_padded(plaintext):
    return unpad(plaintext, 16).decode('ascii')
else:
    return plaintext.decode('ascii')

shared_secret = 143015010881228648
iv = '07ff16c73e99eab4bffbce04e5f30a78'
ciphertext =
'6c4cdfdc516766c869c393e8f0f7c43d1f3ea16304573591c1a577351458330f'

print(decrypt_flag(shared_secret, iv, ciphertext))
```